

Zadania_lab2

Zadanie 1

Napisz funkcję, która wygeneruje tablicę liczb zmiennoprzecinkowych pojedynczej precyzji reprezentujących elementy ciągu postaci:

$$S_n = \sum_{k=0}^{n-1} a_k = \sum_{k=0}^{n-1} \frac{1}{(k \bmod m + 1)(k \bmod m + 2)},$$

gdzie n i m są potęgami liczby 2 oraz $n > m$.

Jak rozumieć to zadanie?

Nie liczymy tu jeszcze całej sumy S_n , tylko najpierw generujemy **same elementy ciągu**:

$$a_k = \frac{1}{(k \bmod m + 1)(k \bmod m + 2)}$$

Czyli dla każdego k :

1. liczysz resztę z dzielenia $k \bmod m$,
2. dodajesz 1 i 2,
3. mnożysz te dwie liczby,
4. bierzesz odwrotność.

Ponieważ zadanie mówi o **pojedynczej precyzji**, każdy element zapisujemy jako `float32`.

Jak rozumieć schemat implementacji?

Krok 1

Tworzysz pustą tablicę.

Krok 2

Dla każdego k od `0` do `n-1` liczysz wartość:

$$a_k = \frac{1}{(k \bmod m + 1)(k \bmod m + 2)}$$

Krok 3

Zapisujesz wynik jako `np.float32`, czyli liczbę pojedynczej precyzji.

Krok 4

Dodajesz element do tablicy.

Kod

PYTHON

```
import numpy as np

def generuj_tablice(n, m):
    tablica = []

    for k in range(n):
        licznik = 1
        mianownik = ((k % m) + 1) * ((k % m) + 2)

        element = licznik / mianownik

        tablica.append(np.float32(element))

    return tablica

n = 64
m = 16

tablica = generuj_tablice(n, m)

print("Elementy ciągu:")
for i, wartosc in enumerate(tablica):
    print("a_", i, "=", wartosc)
```

Co otrzymujemy?

Otrzymujemy tablicę elementów ciągu $(a_0, a_1, a_2, \dots, a_{n-1})$, zapisanych w pojedynczej precyzji.

Zadanie 2

Napisz funkcję sumującą elementy tablicy z zadania 1. Sprawdź dokładność otrzymanej sumy.

Jak rozumieć to zadanie?

Tutaj liczymy już zwykłą sumę wszystkich elementów tablicy:

$$S_n = a_0 + a_1 + a_2 + \cdots + a_{n-1}$$

Jest to najprostszy sposób sumowania:

- zaczynasz od zera,
- dodajesz kolejne elementy jeden po drugim.

Jak rozumieć schemat implementacji?

Krok 1

Generujesz tablicę elementów.

Krok 2

Tworzysz zmienną `suma` równą `0.0` w typie `float32`.

Krok 3

W pętli dodajesz do niej kolejne elementy tablicy.

Krok 4

Na końcu zwracasz wynik.

```
import numpy as np

def generuj_tablice(n, m):
    tablica = []

    for k in range(n):
        licznik = 1
        mianownik = ((k % m) + 1) * ((k % m) + 2)

        a = licznik / mianownik
        tablica.append(np.float32(a))

    return tablica

def sumuj_tablice(tablica):
    suma = np.float32(0.0)

    for element in tablica:
        suma = np.float32(suma + element)

    return suma

n = 64
m = 16

tablica = generuj_tablice(n, m)

suma = sumuj_tablice(tablica)

print("Elementy tablicy:")
for i, wartosc in enumerate(tablica):
    print("a_", i, "=", wartosc)

print("\nSuma elementów:", suma)
```

Jak sprawdzić dokładność?

Dokładną wartość sumy można porównać z:

$$\frac{n}{m+1}$$

W tym zadaniu:

$$\frac{64}{17}$$

To jest wartość odniesienia, z którą porównujemy wynik obliczony numerycznie.

Zadanie 3

Napisz funkcję sumującą elementy tablicy z zadania 1 z wykorzystaniem algorytmu Gilla–Møllera. Sprawdź dokładność otrzymanej sumy.

Co to jest algorytm Gilla–Møllera?

To metoda dokładniejszego sumowania niż zwykłe dodawanie.

W zwykłym sumowaniu część informacji może się gubić przez zaokrąglenia.

Algorytm Gilla–Møllera próbuje zachować tę „zgubioną” część w dodatkowej zmiennej, czyli poprawce.

Jak rozumieć schemat implementacji?

Krok 1

Zaczynasz od:

- `suma = 0`
- `poprawka = 0`

Krok 2

Dla każdego elementu liczysz:

$$t = suma + element$$

Krok 3

Obliczasz, jaka część została utracona podczas dodawania, i dopisujesz ją do poprawki.

Krok 4

Na końcu zwracasz:

$$suma + poprawka$$

```
import numpy as np

def generuj_tablice(n, m):
    tablica = []

    for k in range(n):
        licznik = 1
        mianownik = ((k % m) + 1) * ((k % m) + 2)

        a = licznik / mianownik
        tablica.append(np.float32(a))

    return tablica

def sumuj_tablice(tablica):
    suma = np.float32(0.0)
    poprawka = np.float32(0.0)

    for element in tablica:
        t = np.float32(suma + element)
        poprawka = np.float32(poprawka + (element - (t-suma)))
        suma = t

    return np.float32(suma + poprawka)

n = 64
m = 16

tablica = generuj_tablice(n, m)

suma = sumuj_tablice(tablica)

print("Suma:", suma)
```

Co daje ta metoda?

Wynik zwykle jest dokładniejszy niż przy zwykłym sumowaniu, ponieważ zmniejsza wpływ błędów zaokrągleń.

Zadanie 4

Napisz funkcję sumującą elementy tablicy z zadania 1 z wykorzystaniem algorytmu Kahana. Sprawdź dokładność otrzymanej sumy. Porównaj wyniki wszystkich omówionych metod sumowania.

Co to jest algorytm Kahana?

Algorytm Kahana to inna metoda dokładniejszego sumowania.

Tak samo jak w metodzie Gilla–Møllera, chodzi o to, żeby kontrolować błąd zaokrąglenia. Tutaj używa się zmiennej `c`, która przechowuje utraconą część sumy.

Jak rozumieć schemat implementacji?

Krok 1

Tworzysz:

- `suma = 0`
- `c = 0`

Krok 2

Dla każdego elementu liczysz:

$$y = element - c$$

czyli najpierw korygujesz element o wcześniej utracony błąd.

Krok 3

Dodajesz poprawioną wartość do sumy:

$$t = suma + y$$

Krok 4

Wyznaczasz nowy błąd:

$$c = (t - suma) - y$$

Krok 5

Ustawiasz nową sumę.

```
import numpy as np

def generuj_tablice(n, m):
    tablica = []

    for k in range(n):
        licznik = 1
        mianownik = ((k % m) + 1) * ((k % m) + 2)

        a = licznik / mianownik
        tablica.append(np.float32(a))

    return tablica

def sumuj_tablice(tablica):
    suma = np.float32(0.0)

    for element in tablica:
        suma = np.float32(suma + element)

    return suma

def sumuj_tablice_Møller(tablica):
    suma = np.float32(0.0)
    poprawka = np.float32(0.0)

    for element in tablica:
        t = np.float32(suma + element)
        poprawka = np.float32(poprawka + (element - (t-suma)))
        suma = t

    return np.float32(suma + poprawka)

def sumuj_tablice_Kahan(tablica):
    suma = np.float32(0.0)
    c = np.float32(0.0)

    for element in tablica:
        y = np.float32(element - c)
        t = np.float32(suma + y)
        c = np.float32((t-suma) - y)
        suma = t

    return np.float32(suma)
```

```
n = 64
```



```
m = 16

tablica = generuj_tablice(n, m)

suma = sumuj_tablice(tablica)
suma_m = sumuj_tablice_Møller(tablica)
suma_k = sumuj_tablice_Kahan(tablica)

print("Suma zwykła:", suma)
print("Suma Møller:", suma_m)
print("Suma Kahan:", suma_k)

print("Sprawdzenie dokładności:", n/(m+1))
```

Jak porównać wyniki?

Porównujesz:

- zwykłą sumę,
- sumę metodą Gilla–Møllera,
- sumę metodą Kahana,

z wartością teoretyczną:

$$\frac{n}{m+1}$$

Im bliżej tej wartości, tym metoda jest dokładniejsza.

Wniosek do zadania 4

Zwykle:

- zwykłe sumowanie daje największy błąd,
- metoda Gilla–Møllera daje dokładniejszy wynik,
- metoda Kahana także poprawia dokładność i często daje najlepszy wynik.

Zadanie 5

Przeprowadź analogiczne działania dla danych w podwójnej precyzji.

Co się zmienia?

W poprzednich zadaniach używaliśmy liczb pojedynczej precyzji (`float32`).

Teraz robimy to samo, ale dla liczb podwójnej precyzji, czyli `float` / `float64`.

Podwójna precyzja daje większą dokładność, bo liczba jest przechowywana na większej liczbie bitów.

Jak rozumieć schemat implementacji?

Wszystko działa tak samo jak wcześniej:

- generujesz elementy ciągu,
- sumujesz je trzema metodami,
- porównujesz wyniki.

Różnica polega tylko na tym, że zamiast `np.float32` używasz zwykłego `float`.

```
def generuj_tablice(n, m):
    tablica = []

    for k in range(n):
        licznik = 1
        mianownik = ((k % m) + 1) * ((k % m) + 2)

        a = licznik / mianownik
        tablica.append(float(a))

    return tablica

def sumuj_tablice(tablica):
    suma = float(0.0)

    for element in tablica:
        suma = float(suma + element)

    return suma

def sumuj_tablice_Møller(tablica):
    suma = float(0.0)
    poprawka = float(0.0)

    for element in tablica:
        t = float(suma + element)
        poprawka = float(poprawka + (element - (t-suma)))
        suma = t

    return float(suma + poprawka)

def sumuj_tablice_Kahan(tablica):
    suma = float(0.0)
    c = float(0.0)

    for element in tablica:
        y = float(element - c)
        t = float(suma + y)
        c = float((t-suma) - y)
        suma = t

    return float(suma)
```

n = 64

m = 16

```
tablica = generuj_tablice(n, m)

suma = sumuj_tablice(tablica)
suma_m = sumuj_tablice_Møller(tablica)
suma_k = sumuj_tablice_Kahan(tablica)

print("Suma zwykła:", suma)
print("Suma Møller:", suma_m)
print("Suma Kahan:", suma_k)

print("Sprawdzenie dokładności:", n/(m+1))
```

Jak porównać wyniki z poprzednimi zadaniami?

Porównujesz wyniki z pojedynczej i podwójnej precyzji.

Zwykle:

- w podwójnej precyzji błędy są mniejsze,
- wszystkie metody dają wyniki bliższe wartości teoretycznej,
- metoda zwykła też działa lepiej niż w `float32`, ale nadal może być słabsza od Kahana i Gilla–Møllera.

Wniosek końcowy

W laboratorium porównano różne sposoby sumowania elementów ciągu liczb zmiennoprzecinkowych.

- Zwykłe sumowanie jest najprostsze, ale najbardziej narażone na błędy zaokrągleń.
- Algorytm Gilla–Møllera poprawia dokładność przez przechowywanie poprawki błędu.
- Algorytm Kahana również kompensuje błąd i zwykle daje bardzo dokładne wyniki.
- W podwójnej precyzji wszystkie obliczenia są dokładniejsze niż w pojedynczej precyzji.

Otrzymane wyniki potwierdzają, że sposób sumowania i rodzaj precyzji mają istotny wpływ na dokładność obliczeń numerycznych.